

1. Accidental Modification
2. Thread Safety
3. Debugging Hell
4. Kafka Event
5. Spurious Correlation

Problem Statement

Scenario: "OmniBank Retail Analytics Platform"

You're building a high-performance analytics engine for **OmniBank**, which operates both banking services and a retail chain (OmniMart). The platform must:

1. Process millions of **banking transactions** (deposits, withdrawals, transfers) in real-time
2. Analyze **retail purchases** from OmniMart stores
3. Generate **customer segmentation reports** (premium, gold, silver tiers)
4. Compute **fraud detection statistics** using custom aggregations
5. Optimize memory for handling **10M+ records** without OutOfMemoryError
6. Ensure **thread-safe immutable DTOs** for audit compliance
7. Pool expensive objects (currency formatters, encryption engines)
8. Protect sensitive data from accidental mutation

name: Rahul
 account: 1234r
 balance: 10000
 0

Airport baggage
 CRIS = 101
 amt = 200.0
 Bad
 amt = 1

Use records for DTOs in Java

Traditional POJO DTO

```
public class ProductDto {
    private String name;
    private BigDecimal price;
    private int stock;

    public ProductDto(String name, BigDecimal price, int stock) {
        if (name == null || name.isBlank())
            throw new IllegalArgumentException("Name cannot be null");
        if (price == null || price.compareTo(BigDecimal.ZERO) <= 0)
            throw new IllegalArgumentException("Invalid price");

        this.name = name;
        this.price = price;
        this.stock = stock >= 0 ? stock : 0;
    }

    public String getName() { return name; }
    public BigDecimal getPrice() { return price; }
    public int getStock() { return stock; }
}
```

Modern Record with Compact Constructor

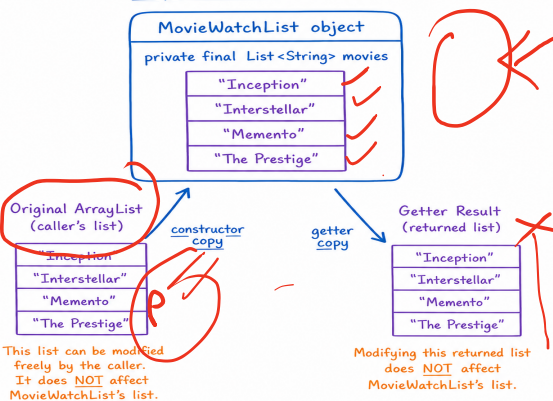
```
public record ProductDto(String name, BigDecimal price, int stock) {
    public ProductDto {
        if (name == null || name.isBlank())
            throw new IllegalArgumentException("Name cannot be null");

        if (price == null || price.compareTo(BigDecimal.ZERO) <= 0)
            throw new IllegalArgumentException("Invalid price");

        stock = stock >= 0 ? stock : 0;
    }
}
```

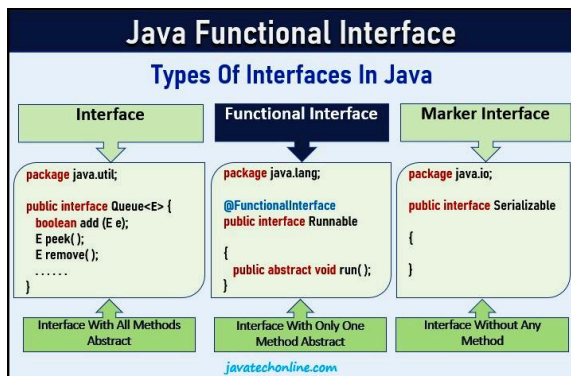
Defensive Copying in Java

MovieWatchList makes copies to protect its internal list.

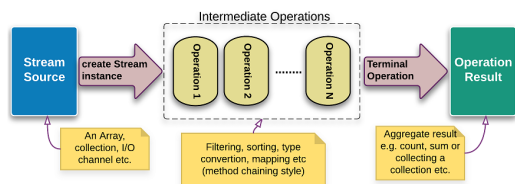


MovieWatchList's internal list is private and cannot be modified from outside

and cannot be modified from outside.
All access is safely copied.



Java Streams



Java Streams

